IMPORT LIBRARIES

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDis
from sklearn.decomposition import PCA
```

LOAD AND PREPARE THE DATASET

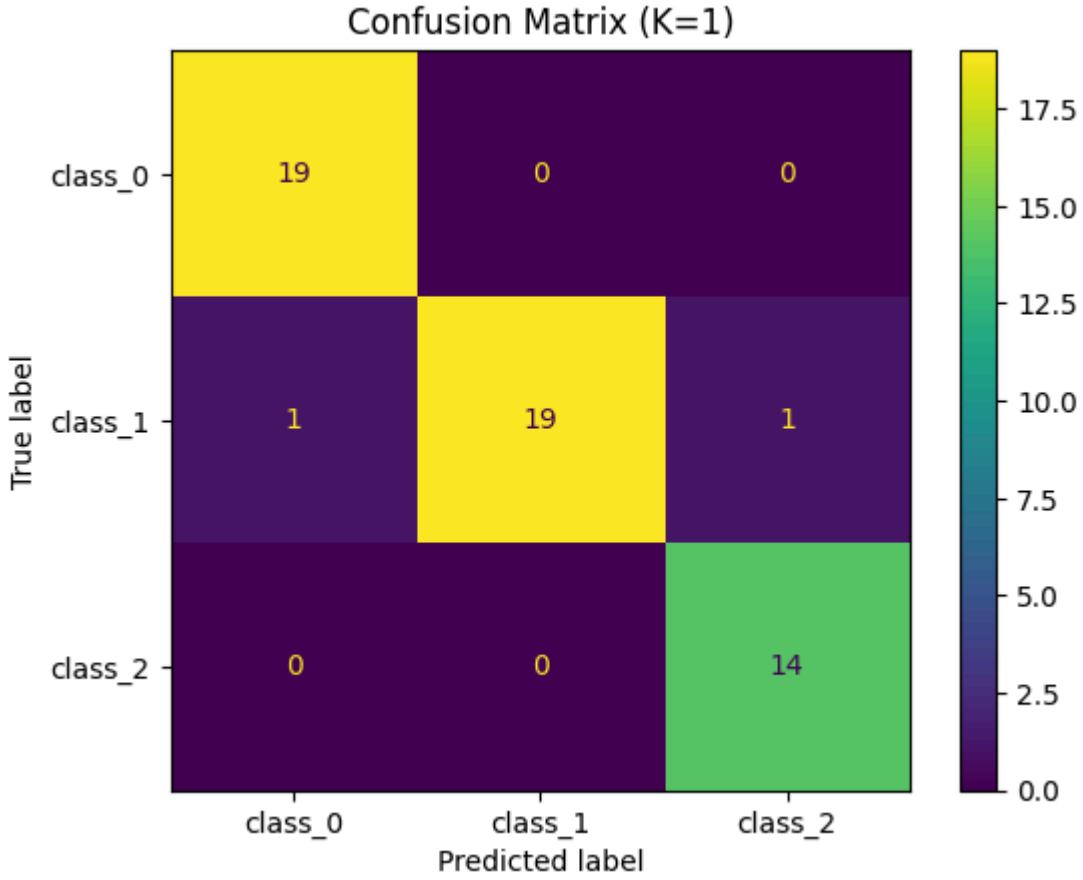
In [3]:
```
wine = load_wine()
X = wine.data
y = wine.target
feature_names = wine.feature_names
target_names = wine.target_names
```

In [4]:
```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.3, random_s
# Normalize the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

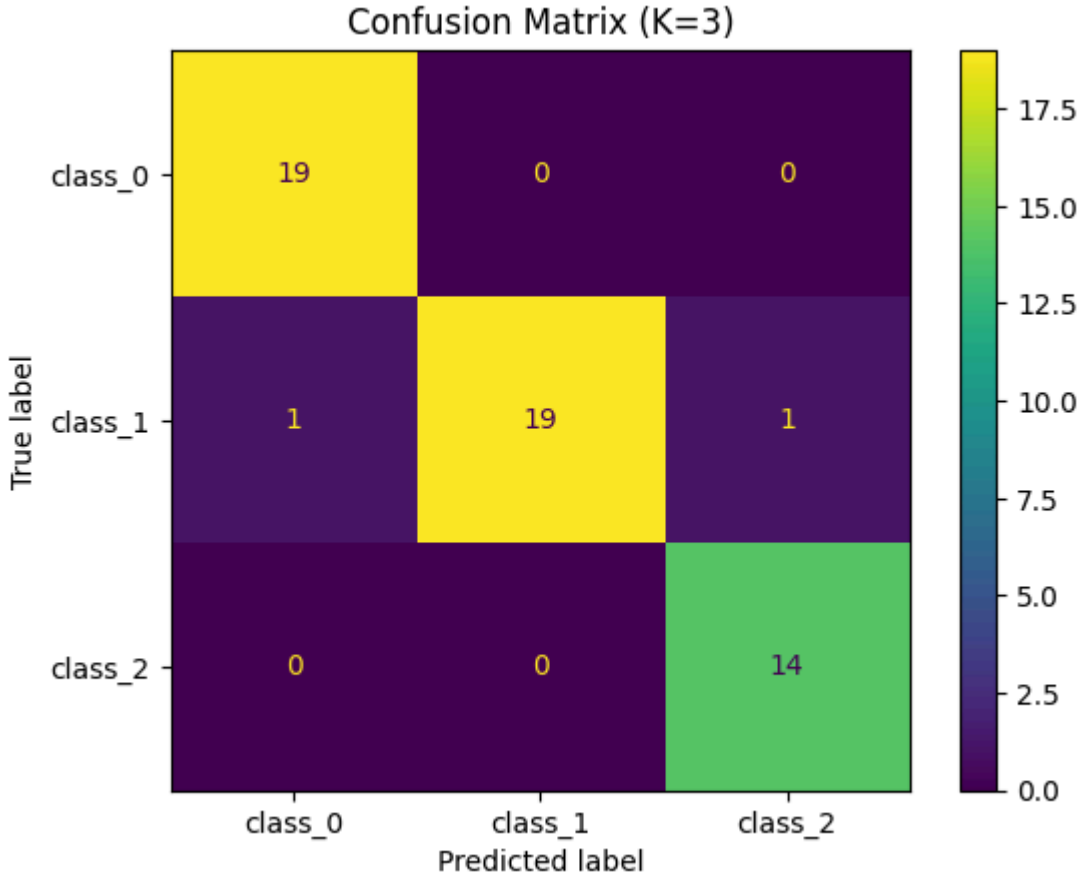
TRAIN THE KNN AND EVALUATE FOR DIFFERENT K

In [5]:
```
k_values = [1,3,5,7]
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)
    print(f"\nK = {k}")
    print(f"Accuracy = {acc:.2f}")

# Confusion Matrix
    cm = confusion_matrix(y_test,y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_nam
    disp.plot()
    plt.title(f'Confusion Matrix (K={k})')
    plt.show()
```

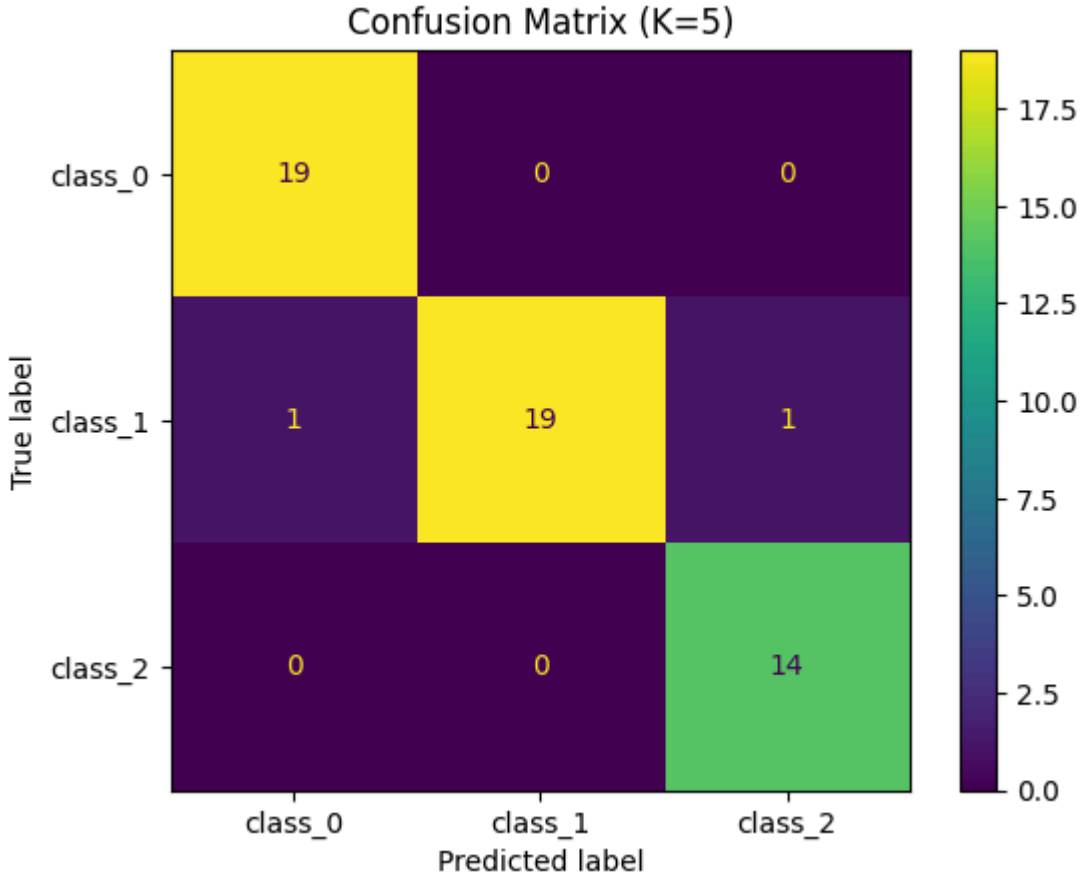
```
K = 1
Accuracy = 0.96
```



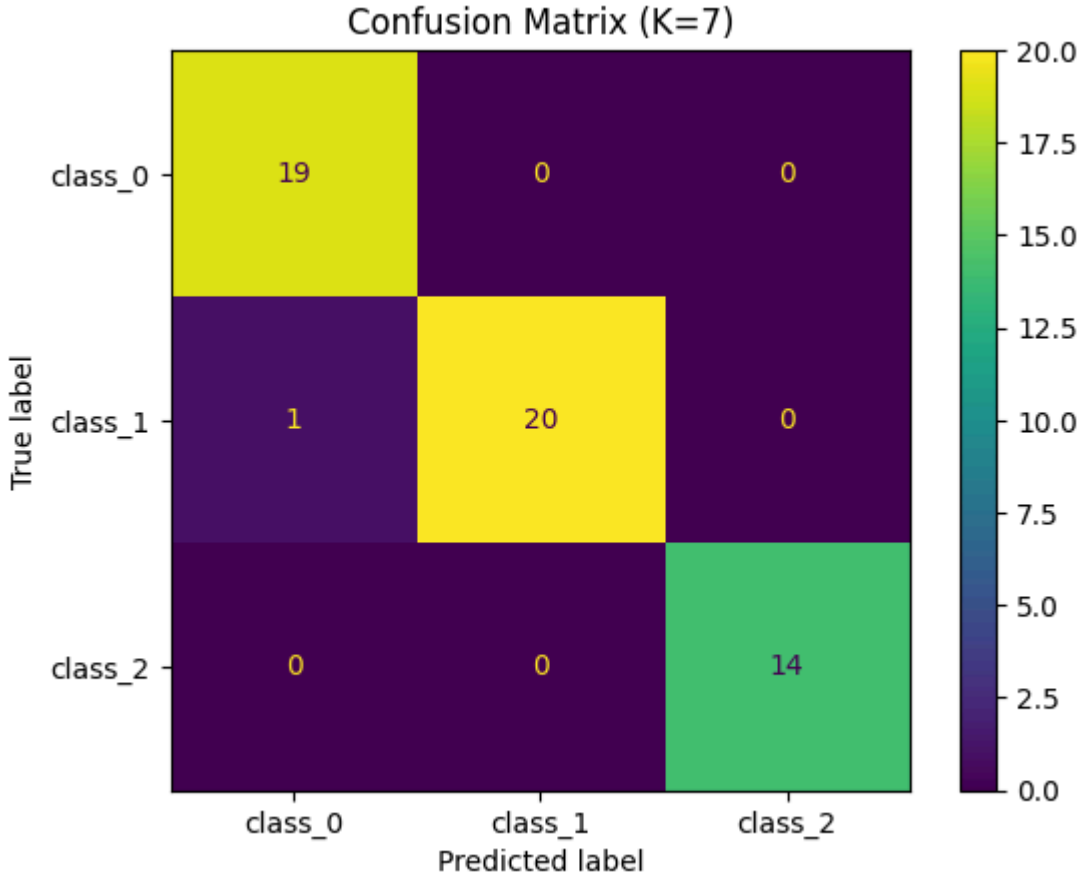

```
K = 3
Accuracy = 0.96
```




```
K = 5
Accuracy = 0.96
```

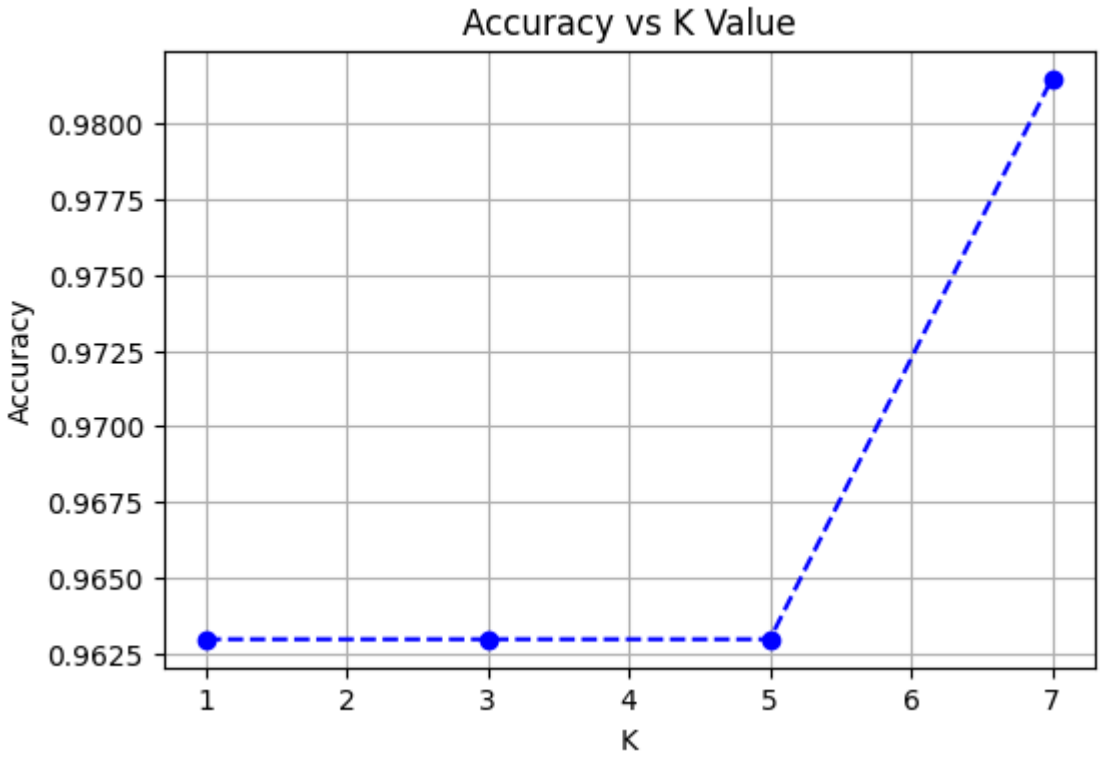



```
K = 7
Accuracy = 0.98
```



In [6]:
```
plt.figure(figsize=(6,4))
plt.plot(k_values, accuracies, marker='o', linestyle='--', color='b')
plt.title('Accuracy vs K Value')
```

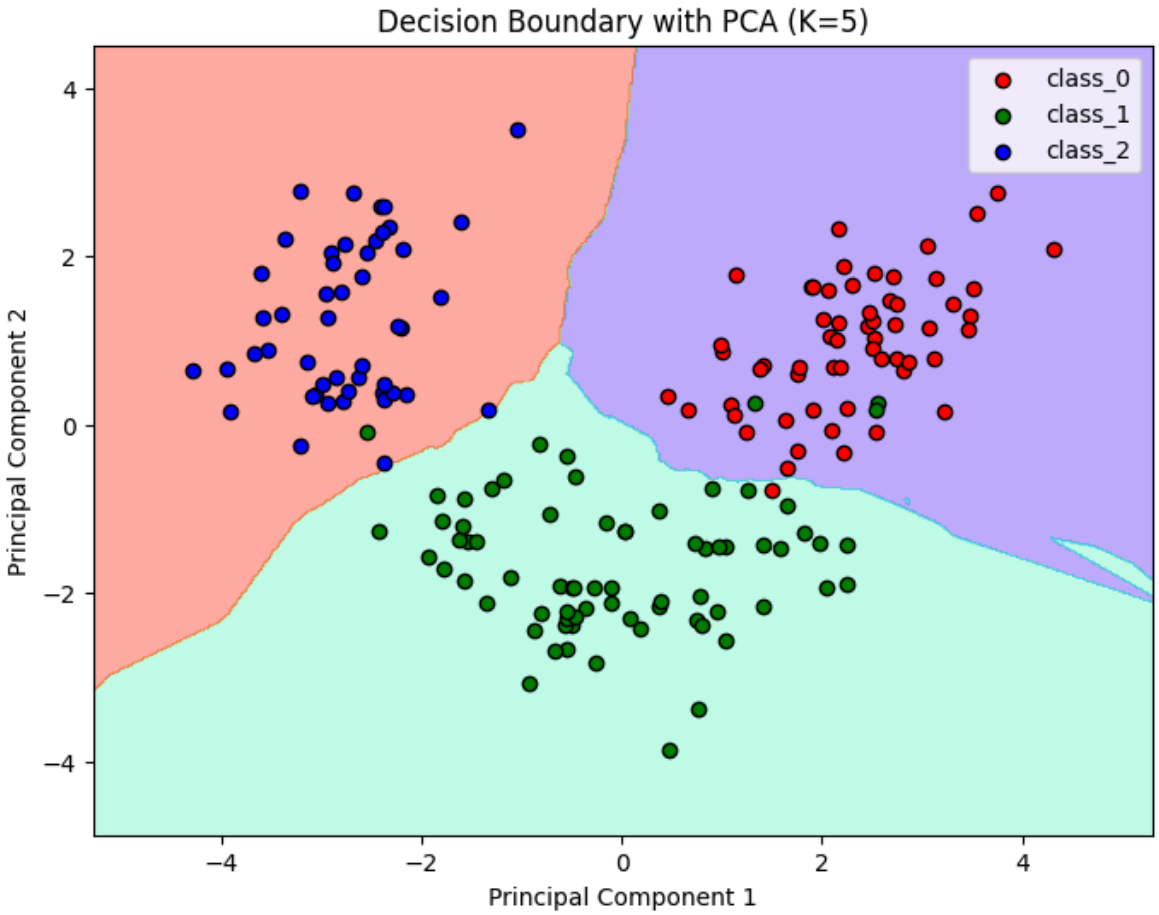
```
plt.xlabel('K')
plt.ylabel('Accuracy')
plt.grid(True)
plt.show()
```



VISUALIZE DECISION BOUNDARIES USING PCA

In [8]:
```
# Reduce to 2D using PCA for visualization
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(scaler.fit_transform(X))
X_train_pca, X_test_pca, y_train, y_test = train_test_split(X_reduced, y, test_s

# KNN on PCA-reduced data
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_pca, y_train)

# Create mesh for decision boundary
h = .02
x_min, x_max = X_reduced[:, 0].min() - 1, X_reduced[:, 0].max() + 1
y_min, y_max = X_reduced[:, 1].min() - 1, X_reduced[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.4, cmap=plt.cm.rainbow)

colors = ['red', 'green', 'blue']
for i, color in zip(np.unique(y), colors):
    plt.scatter(X_reduced[y == i, 0], X_reduced[y == i, 1],
                color=color, label=target_names[i], edgecolor='k')

plt.title("Decision Boundary with PCA (K=5)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
```

```
plt.legend()
plt.show()
```



In [ ]: